

Feedback-Efficient Semantic Repair for LLM-Generated Optimization Code

Peiran Niu

Massachusetts Institute of Technology

niup683@mit.edu

MAS.S60 / 6.S985 Multimodal AI Final Project

Project repository: github.com/niu1298/.../feedback_efficiency_experiment

Abstract

Large language models make optimization autoformulation attractive because they can translate natural-language decision problems into Python/Gurobi code, but executable code can still encode the wrong mathematical model. Execution feedback is a strong evaluator, yet it is often too coarse for repair because an objective mismatch does not explain which constraint, variable domain, or objective term is wrong. I study semantic advisory feedback for language-to-Gurobi repair on the 92-problem LogiOR benchmark. A second model reads the original problem and candidate code, writes a modeling diagnosis, and adds that diagnosis to the next repair prompt without blocking execution. Semantic advisory helps most when the advisor has a capability advantage over the generator: with **gpt-5-mini** advisory, **gpt-4o-mini** improves from 31/92 to 56/92 solved problems. In the final repair comparison, execution-only solves 25/92 problems, direct semantic advisory solves 46/92, adaptive compression solves 37/92, and formulation-spec-before-code, denoted **spec_then_code**, solves 44/92. Efficiency analysis gives the same conclusion a practical interpretation: on the hardest band, advisory reduces **gpt-4o-mini** API calls per solved problem from 36.3 to 19.2, and adaptive compression reduces maximum feedback length from 9,915 to 3,000 characters while still outperforming execution-only repair. Retrospective gating shows execution-saving potential, but false rejections make hard gating unsafe. The main lesson is that semantic checking is useful as diagnosis-only repair context, not as a formal verifier or hard gate.

1 Introduction

Mathematical optimization is a standard language for decision-making problems in logistics, production planning, routing, finance, scheduling, and resource allocation. In many applications, the hard step is not running a solver after the model is written. The hard step is translating a human description into the right decision variables, objective, constraints, and domains. Recent logistics-focused work makes the same point: optimization modeling has a high expertise threshold, and richer benchmarks such as LogiOR are needed to evaluate whether models can handle realistic formulation logic [Yang et al., 2025].

Large language models make optimization autoformulation attractive because they can produce solver code directly from natural language. NL4Opt, OptiMUS, ORLM, LLMOPT, Chain-of-Experts, and related systems show that this is now a real research direction rather than a small demonstration [Ramamonjison et al., 2022, AhmadiTeshnizi et al., 2024, Tang et al., 2024, Jiang et al., 2024, Xiao et al., 2024b, Astorga et al., 2024]. This line of work shifts the central question from “can the model write code?” to “does the generated code faithfully encode the intended mathematical program?”

Execution feedback is necessary, but it is not a full semantic explanation. Solver-backed systems can catch syntax errors, API misuse, infeasibility, missing objective output, and objective mismatch. EVOM, for example, treats the mathematical programming solver as a deterministic feedback source in a generate-execute-feedback-update loop [Guan et al., 2026]. That makes execution a strong baseline. However, a wrong objective value only says that the candidate failed; it does not say whether the model omitted a balance constraint, relaxed an integer variable, flipped an inequality, double-counted revenue, or reported the wrong quantity.

A useful way to frame the missing piece is through verification. In verified code generation, formal verifiers can provide reliable correctness signals for self-play and self-improvement; PSV is a recent example of this idealized setting [Wilf et al., 2025]. Optimization autoformulation usually lacks that kind of verifier. The solver verifies the generated mathematical program, not the semantic translation from the user’s text to the formulation. There is no simple Verus-like oracle for “this Gurobi code faithfully expresses the natural-language model.”

Semantic checking is a practical middle layer between execution-only repair and formal verification. SAC-Opt is closely related in spirit: it argues that optimization modeling should compare generated code against problem semantics rather than rely only on solver feedback, using semantic anchors and correction to refine mismatched objectives and constraints [Zhang et al., 2025b]. My project follows the same motivation but uses an API-based semantic advisor inside an inference-time repair loop. I initially explored a local semantic verifier, but the available paired data and near-miss negatives were not sufficient to make it reliable enough for the final repair loop. I therefore use semantic advisory as a practical feedback signal rather than as a learned verifier.

The advisor reads the original problem and generated code, then returns diagnosis text about missing constraints, wrong objective terms, variable/domain issues, Gurobi API risks, and output-format errors. Crucially, this diagnosis is repair context, not a hard gate. Every candidate is still executed by the harness. The advisor only helps construct the next repair prompt, preserving execution as the final evaluator while adding formulation-level feedback that execution alone does not provide.

A small example shows the motivation. In LogiOR `prob_037`, the generated code was executable and solved to optimality, but it counted revenue from ordinary products and premium products even when ordinary units were converted into premium units. Execution reported only an objective mismatch. The semantic advisor named the modeling error—missing flow equalities and double-counted revenue—which made the next repair much more targeted.

This report asks two empirical questions. First, when does semantic advisory feedback help repair generated optimization code? Second, once advisory is useful, how should semantic feedback be packaged efficiently? I evaluate advisory across problem difficulty, generator capability, and judge capability, then compare direct advisory with adaptive feedback compression and formulation-spec-before-code, denoted `spec_then_code`.

Contributions. First, I show that `gpt-5-mini` advisory raises `gpt-4o-mini` from 31/92 to 56/92 solved LogiOR problems, with +25 to +30 percentage-point gains across all difficulty bands. Second, I show that the same advisory is neutral for `gpt-5-mini`: 53/92 with execution-only repair, 53/92 with same-model advisory, and 54/92 with `kimi-k2.6` advisory. Third, I compare final repair designs and find that direct advisory solves 46/92 problems, `spec_then_code` solves 44/92, adaptive compressed advisory solves 37/92, and execution-only solves 25/92. Fourth, I analyze why diagnosis text is safer and more useful than hard gating.

2 Related Work

Natural-language optimization and autoformulation. NL4Opt framed natural-language optimization as a benchmarked autoformulation task [Ramamonjison et al., 2022]. Later systems explored multi-role prompting and modular reasoning [AhmadiTeshnizi et al., 2024, Xiao et al., 2024b], supervised fine-tuning for optimization modeling [Tang et al., 2024], search over candidate formulations [Huang et al., 2024a], and general LLM autoformulation pipelines [Astorga et al., 2024, Jiang et al., 2024]. ORThought and LogiOR emphasize that harder logistics problems require expert-style formulation reasoning and more diagnostic evaluation [Yang et al., 2025]. These systems share the same core difficulty: textual constraints must be converted into a precise mathematical structure. Benchmark work has also expanded the evaluation setting. Mamo uses solver-backed mathematical modeling tasks to make end-to-end correctness measurable, while OptiBench and ReSocratic study both measurement and data synthesis for optimization modeling [Huang et al., 2024b, Yang et al., 2024]. Recent surveys place LLM-based optimization modeling inside a broader movement from static prompting toward self-evolving optimization agents [Zhang et al., 2025c].

Execution-verified repair. Execution-verified optimization modeling uses the solver as a reliable external check. EVOM makes this view explicit by treating a solver as a deterministic interactive verifier and converting execution outcomes into rewards for optimization-modeling training [Guan et al., 2026]. Related work in mathematical and code reasoning also uses execution or reward signals to improve generated solutions [Shao et al., 2024]. In this project, Gurobi [Gurobi Optimization, LLC, 2026] provides deterministic feedback about compile errors, runtime failures, solver status, and objective mismatches. That feedback is essential, but it is semantically sparse. Recent self-improving OR systems extend this loop in different ways. StepORLM addresses the credit-assignment weakness of outcome-only feedback with generative process supervision, while AlphaOPT maintains an external experience library that distills lessons from previous failures without updating model weights [Zhou et al., 2025, Kong et al., 2025]. My work does not train a policy model or maintain a learned experience library; it studies whether semantic diagnosis can be used directly as inference-time repair context.

Verification and semantic correction. Formal verification provides an idealized contrast: PSV shows that reliable verifiers can support self-play from correctness feedback [Wilf et al., 2025]. Optimization autoformulation has a weaker signal, because execution verifies program behavior rather than semantic faithfulness to text. SAC-Opt is the closest related direction because it targets semantic alignment between problem statements and generated solver code [Zhang et al., 2025b]. Local semantic verification and encoders such as BGE or MathBERT remain background motivation here; the implemented system uses API-based semantic diagnosis during repair [Xiao et al., 2024a, Peng et al., 2021].

LLM self-repair and tool feedback. The repair loop is also related to Reflexion, ReAct, Self-Refine, and self-debugging work [Shinn et al., 2023, Yao et al., 2022, Madaan et al., 2023, Chen et al., 2023]. Those methods show that language feedback can help a model revise its own output. OPTIMIND is close in motivation because it injects optimization-expert hints and multi-turn correction into the modeling process [Zhang et al., 2025a]. In contrast, the advisor here is not a class-specific hint library; it reads the generated code and problem instance at each round and produces instance-specific formulation diagnostics.

3 Problem Statement

Let x be a natural-language optimization problem, c_t the generated code at repair round t , e_t the execution feedback, d_t a semantic diagnosis, and s a structured formulation spec. Let G be the code generator, E the execution harness, A the semantic advisor, C a compression policy, and P a spec generator.

At each repair round, the system has a candidate program c_t . The execution harness maps this candidate to feedback $e_t = E(c_t)$, such as a runtime error, `no_objective` output, solver status, or objective mismatch. Execution-only repair feeds x , c_t , and e_t back to the generator to obtain c_{t+1} . Semantic advisory adds a second transition: the advisor maps the same problem and candidate code to a diagnosis $d_t = A(x, c_t)$, and the next prompt contains both execution feedback and semantic diagnosis.

The repair transitions are:

$$e_t = E(c_t), \quad c_{t+1} = G(x, c_t, e_t) \quad \text{execution-only,} \quad (1)$$

$$d_t = A(x, c_t), \quad c_{t+1} = G(x, c_t, e_t, d_t) \quad \text{direct advisory,} \quad (2)$$

$$\tilde{d}_t = C(d_t), \quad c_{t+1} = G(x, c_t, e_t, \tilde{d}_t) \quad \text{compressed advisory,} \quad (3)$$

$$s = P(x), \quad c_0 = G(x, s) \quad \text{spec_then_code.} \quad (4)$$

Operationally, each method defines a repair policy π . I evaluate π by the number of LogiOR problems solved within the allowed repair horizon T :

$$S(\pi) = \sum_{i=1}^{92} \mathbf{1}\{\text{valid}(E(c_T^{(i)}))\}. \quad (5)$$

Here valid means that the generated program executes and prints an objective matching the evaluator reference within tolerance. The advisor also emits structured JSON, including a `should_execute` flag, but

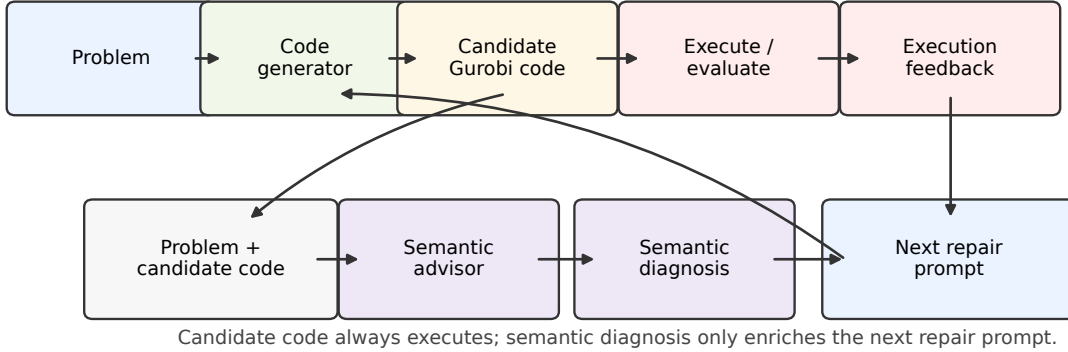


Figure 1: Core semantic repair loop. Execution remains the evaluator. The advisor reads the problem and candidate code, then adds diagnosis text to the next repair prompt; it never blocks execution. Adaptive compression and `spec_then_code` are method variants described in text.

I use this flag only for analysis. The repair loop still executes every candidate program, because a false rejection can hide a correct solution.

4 Method

Component	Used in	Role
Code generator	all methods	writes Python/Gurobi code
Execution harness	all methods	runs code, parses objective, validates against reference
Semantic advisor	direct/adaptive advisory	reads problem + code and diagnoses formulation errors
Adaptive compression	adaptive compressed only	shortens diagnosis before repair
Spec generator	<code>spec_then_code</code> only	writes structured formulation plan
Formulation spec	<code>spec_then_code</code> only	variables/objective/constraints/domains/output

Table 1: The components are not all active at once. Direct advisory uses the code generator, execution harness, and semantic advisor; `spec_then_code` uses a separate spec-generation step before code generation.

Execution-only baseline. The baseline generates Python/Gurobi code from the problem, executes it, parses the printed objective, and repairs from the harness feedback. The next prompt contains compile errors, runtime errors, missing-objective messages, solver status, or objective mismatch information, but no semantic advisor output.

Direct semantic advisory. The semantic advisor is a separate LLM call. It reads the original problem and current generated code, then returns diagnosis text about the formulation. It does not generate the final code and does not block execution. Execution feedback can say that the objective is wrong; semantic advisory tries to explain why it is wrong.

The diagnosis acts as a compact repair contract: suspected modeling error, evidence from the candidate code, concrete repair instruction, and output/API warning when relevant. Typical mappings are: missing constraint $\rightarrow$ add a balance, capacity, or flow-conservation relation; wrong objective $\rightarrow$ remove double counting, fix the sign, or add a missing cost/revenue term; variable/domain issue $\rightarrow$ introduce the missing flow, sold, binary, or integer variable; output issue $\rightarrow$ print exactly one numeric objective in the expected format; API/runtime issue $\rightarrow$ fix invalid Gurobi construction. The `should_execute` flag is logged for analysis but never used to block execution.

Adaptive compressed advisory. Full advisory can become verbose. Adaptive compression keeps high-value content first: missing constraints, wrong objective terms, named variables and constraints, run-time/API guidance after execution failures, and output guidance after `no_objective`. It removes repetition and lower-value generic comments. The goal is prompt efficiency, not a claim that compressed feedback is more informative than full advisory.

Formulation-spec-before-code. `spec_then_code` is a separate variant, not part of the direct-advisory loop in Figure 1. It first calls a spec generator to produce a formulation spec—decision variables, objective, constraints, domains, and output requirements—and then asks the code generator to write Gurobi code from the problem plus that spec. This shifts part of the burden from post-hoc repair to initial planning. In the final comparison, `spec_then_code` solves 44/92 problems with 308 LLM calls and 154 solver calls.

Analysis-only modules. I also evaluated code-to-spec extraction, intended-vs-extracted spec comparison, static/spec/semantic hybrid diagnosis, multi-advisor disagreement, feedback uptake, and retrospective gate policies. These modules are not part of the final repair loop. They are useful mainly for understanding why advisory works and why hard gating is risky.

5 Experimental Methodology

The experiments are organized around two questions. The first experiment is a cross-difficulty semantic-advisory study: it asks whether semantic feedback helps at all, and how the answer depends on generator capability, judge capability, and problem difficulty. This experiment uses a five-round repair horizon and compares execution-only repair with diagnosis-only advisory across `gpt-4o-mini`, `gpt-5-mini`, and `kimi-k2.6`.

The second experiment is a refined repair-system comparison. After the advisory study showed that semantic feedback can be useful, I fixed a tighter two-round repair horizon and compared different ways to package semantic information: full direct advisory, adaptive compressed advisory, and formulation-spec-before-code, denoted `spec_then_code`. This setting is more constrained in repair budget, so its solve counts should not be directly compared to the five-round advisory study. It is intended to compare final system designs under a common protocol.

Regime	Main question	Key setting
Experiment 1: cross-difficulty advisory study	Does semantic advisory help across model capability and difficulty?	Five-round horizon; two generators; same-provider and cross-provider judges.
Experiment 2: refined repair-system comparison	How should semantic feedback be packaged in the final repair loop?	Stricter two-round horizon; fixed variants with <code>gpt-4o-mini</code> generation and <code>gpt-5-mini</code> semantic/spec calls.

Table 2: Experimental regimes. The two experiments answer different questions. Experiment 2 is stricter in repair horizon, not necessarily harder in every possible sense. The 56/92 and 46/92 results should not be interpreted as a regression.

5.1 Experiment 1: Cross-Difficulty Semantic Advisory Study

The advisory study asks whether semantic feedback helps, and when. It uses all 92 LogiOR problems. I sort problems by difficulty = `variables_num` + `constraints_num` and split them into four bands: Band 1 has 20 problems with difficulty 3–15, Band 2 has 20 problems with difficulty 16–29, Band 3 has 20 problems with difficulty 30–66, and Band 4 has 32 problems with difficulty 69–5274.

The generators are `gpt-4o-mini` and `gpt-5-mini`. The judges are `gpt-5-mini` and `kimi-k2.6`. The five configurations are `gpt-4o-mini` execution-only, `gpt-4o-mini` with `gpt-5-mini` advisory, `gpt-5-mini` execution-only, `gpt-5-mini` with same-model advisory, and `gpt-5-mini` with `kimi-k2.6` advisory. Each run uses temperature 0.0, a 2,048-token generation budget, and up to five repair rounds. The expected objective is masked from generator prompts.

Generator / Strategy	Band 1	Band 2	Band 3	Band 4
<code>gpt-4o-mini</code> / execution-only	10/20 (50%)	9/20 (45%)	8/20 (40%)	4/32 (12%)
<code>gpt-4o-mini</code> / advisory(<code>gpt-5-mini</code>)	16/20 (80%)	14/20 (70%)	13/20 (65%)	13/32 (41%)
<code>gpt-5-mini</code> / execution-only	16/20 (80%)	13/20 (65%)	16/20 (80%)	8/32 (25%)
<code>gpt-5-mini</code> / advisory(<code>gpt-5-mini</code>)	16/20 (80%)	12/20 (60%)	16/20 (80%)	9/32 (28%)
<code>gpt-5-mini</code> / advisory(<code>kimi-k2.6</code>)	15/20 (75%)	12/20 (60%)	16/20 (80%)	11/32 (34%)

Table 3: Cross-difficulty advisory results on LogiOR. Advisory gives the largest gains for the weaker generator; the stronger generator changes little except on the hardest band.

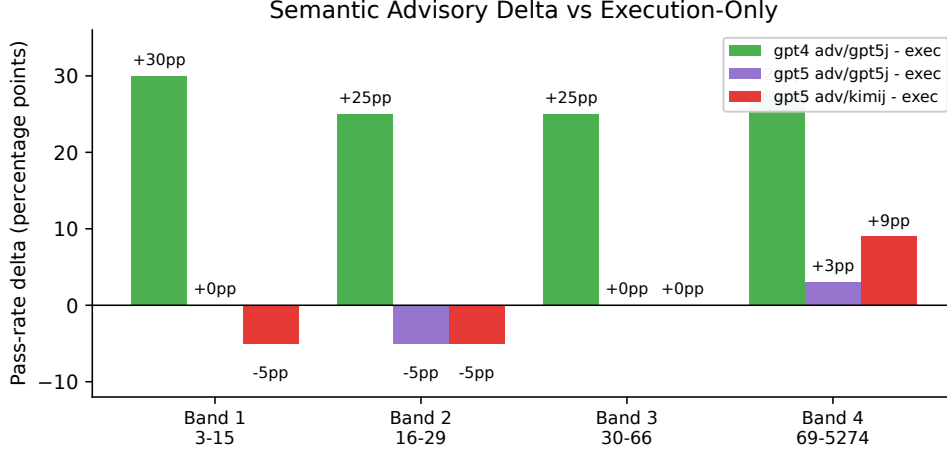


Figure 2: Advisory pass-rate deltas relative to same-generator execution-only runs. The `gpt-4o-mini` gains are +30, +25, +25, and +28 percentage points from Band 1 to Band 4.

The primary metric is solve rate. I also report paired rescue/loss counts, exploratory Fisher and exact paired tests, judge precision/recall/F1 using execution validity as ground truth, retrospective hard-gate behavior, and API calls per solved problem.

5.2 Experiment 2: Refined Semantic Repair-System Comparison

The final comparison asks how to package semantic feedback once advisory has proved useful. It compares four methods on the same 92 problems: execution-only baseline, original direct semantic advisory, adaptive compressed advisory, and `spec_then_code`. The code generator is `gpt-4o-mini`; semantic and spec calls use `gpt-5-mini`. The repair horizon is two rounds, temperature is 0.0, and the solver timeout is 30 seconds.

The reported metrics are solved count, pass rate, solver calls, LLM calls, parse success where applicable, maximum prompt length, maximum feedback length, safety counters, per-round convergence, and solve overlap. The final comparison does not include Kimi, hard gating, trained local verification, or full multi-advisor repair.

6 Results and Discussion

I report the results in the same order. Sections 6.1 and 6.2 establish the advisory effect and the types of errors it repairs. Section 6.3 then analyzes call efficiency from the advisory study together with bounded-horizon convergence and prompt length from the final comparison. Section 6.4 compares final semantic packaging choices under the stricter two-round protocol.

gpt-4o-mini execution-only vs. semantic advisory

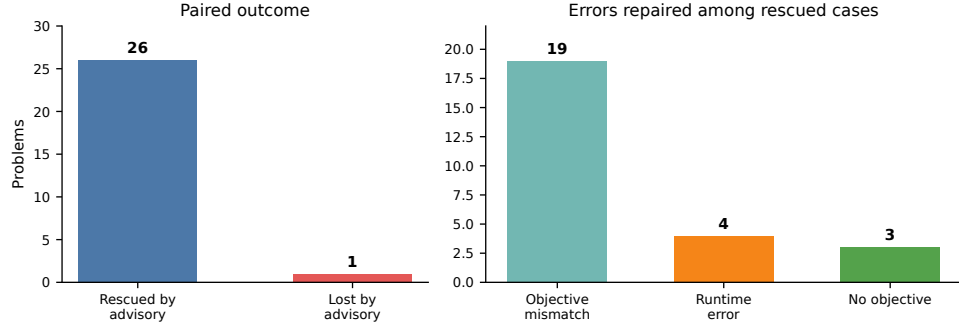


Figure 3: Paired rescue/loss and rescued-error breakdown for `gpt-4o-mini` execution-only versus `gpt-5-mini` advisory. Most rescued cases were objective mismatches, which is the failure mode most closely tied to semantic formulation errors.

6.1 Experiment 1: Semantic Advisory Helps When the Judge Adds Capability

Table 3 and Figure 2 show the central pattern. For `gpt-4o-mini`, advisory improves every band: +30pp, +25pp, +25pp, and +28pp. Pooled over all 92 problems, execution-only solves 31/92 and advisory solves 56/92, a +27.17 percentage-point improvement. The paired view is more revealing: advisory rescues 26 previously failed problems and loses only 1 previous success, with exact paired $p = 4.17 \times 10^{-7}$.

The stronger generator does not show the same benefit. `gpt-5-mini` solves 53/92 with execution-only repair, 53/92 with same-model advisory, and 54/92 with `kimi-k2.6` advisory. The negative result is useful because it shows that advisory is not automatically helpful. A good diagnosis must add information the generator does not already have.

6.2 Experiment 1: Advisory Repairs Semantic Formulation Errors

The paired comparison clarifies what semantic feedback is doing. Advisory rescues 26 problems that execution-only failed and loses only 1 problem that execution-only solved. The rescued execution-only errors include 19 objective mismatches, 4 runtime errors, and 3 `no_objective` failures (Figure 3). The largest group of rescued cases were objective mismatches: 19 of the 26 `gpt-4o-mini` rescues. This is exactly the regime where execution says the code is wrong but not why.

Example 1: LogiOR prob_037, product-flow repair. In `prob_037`, the generated code executed but returned 162,990 instead of the reference 156,500. The underlying issue was semantic: the code treated ordinary and premium products as independently sellable and counted converted units twice. The advisor identified missing flow equalities and double-counted revenue, then recommended sold variables or equality constraints linking production, conversion, and sales. The next repair added sold variables, enforced flow conservation, rewrote revenue on sold quantities, and returned the correct objective 156,500. This example shows why semantic feedback is useful: it turns an unexplained wrong objective into concrete formulation edits. Appendix D gives the compact code-level trace.

6.3 Operational Efficiency and Bounded-Horizon Convergence

Beyond solve rate, I evaluate three operational quantities: API calls per solved problem, prompt/feedback character length, and retrospective execution savings from a hypothetical gate. These metrics are not dollar costs, but they show whether semantic feedback is practical under bounded repair budgets. API/Solved is total API calls divided by solved problems, where lower is better. Prompt and feedback lengths are measured in characters, not tokens or dollars. Gate metrics use retrospective `should_execute` judgments: Saved % measures solver-call saving, rejection accuracy measures whether skipped candidates were actually bad, and

false rejection rate measures solved candidates that would have been wrongly rejected. Appendix Table 8 gives the full definitions.

Advisory is not free, because each advisory round adds an LLM call. The relevant question is whether the added calls buy enough successful repairs. The advisory API counts include semantic-check/advisor calls. Each generator or advisor request is counted as one API call; calls are not weighted by token count, latency, or model price. On Band 4, **gpt-4o-mini** execution-only uses 145 API calls for 4 solves, or 36.3 calls per solve. Advisory uses 249 API calls for 13 solves, or 19.2 calls per solve. Thus advisory uses more total calls but fewer calls per solved problem on Band 4. For the stronger generator, **gpt-5-mini** execution-only uses 16.0 calls per solve on Band 4, while **gpt-5-mini** with **kimi-k2.6** advisory uses 12.4 (Figure 4). These are API-call counts, not token or dollar costs. The metric improves when the additional judge calls are outweighed by the increase in solved hard problems.

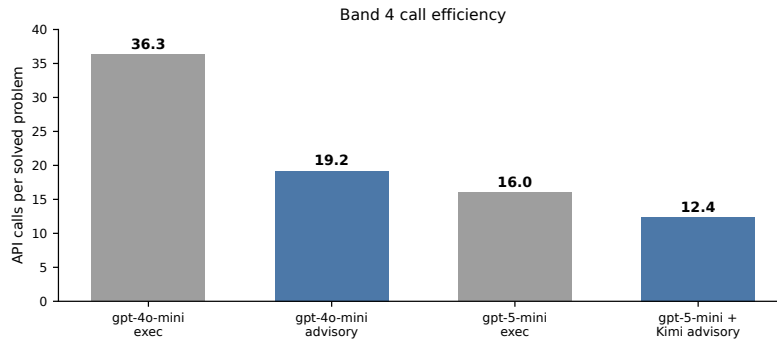


Figure 4: Band 4 call efficiency in the cross-difficulty advisory study. The y-axis is API calls per solved problem, so lower is better. The advisory API counts include semantic-check/advisor calls. Each generator or advisor request is counted as one API call; calls are not weighted by token count, latency, or model price. Advisory uses more total calls but fewer calls per solved problem on Band 4 because the additional judge calls are outweighed by more solved hard instances.

The final comparison shows a different kind of efficiency: convergence within a two-round horizon. Original advisory moves from 20 initial solves to 46 final solves, the largest repair jump. **spec_then_code** starts at 30 initial solves and ends at 44. It starts higher because it performs formulation planning before code generation. It is not a standard zero-shot code generation baseline; its first code attempt is conditioned on a structured spec containing variables, objective, constraints, domains, and output requirements (Figure 5). Adaptive compression improves from 19 to 37 and shortens the feedback contract, but it loses some repair signal relative to full advisory.

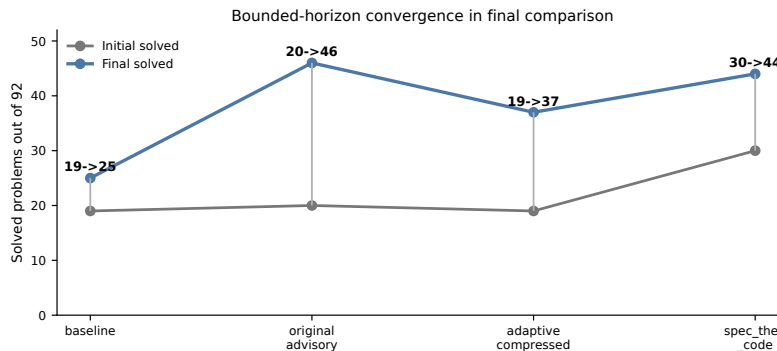


Figure 5: Convergence in the final repair-system comparison. The x-axis is method and the y-axis is solved problems out of 92, so higher is better. Initial solved is the first generated candidate under that method. For **spec_then_code**, that first candidate is conditioned on a formulation spec, explaining its stronger initial count. Direct advisory has the largest repair jump.

Original advisory is verbose because it includes full diagnosis. Adaptive compressed advisory shortens feedback from 9,915 to 3,000 characters and maximum prompt length from 17,952 to 10,667, while still solving 37/92 problems versus 25/92 for execution-only. Appendix Figure 12 visualizes this prompt-length tradeoff.

6.4 Experiment 2: Final Repair-System Comparison

Method	Solved	Pass Rate	Solver Calls	LLM Calls	Parse	Max Prompt	Max Feedback	Leaks	Containers
baseline	25/92	0.272	165	165	n/a	12277	3687	0	0
original advisory	46/92	0.500	164	328	0.988	17952	9915	0	0
adaptive compressed	37/92	0.402	165	330	0.988	10667	3000	0	0
<code>spec_then_code</code>	44/92	0.478	154	308	0.981	15791	3803	0	0

Table 4: Final repair-system comparison on all 92 LogiOR problems. Direct advisory is strongest; `spec_then_code` is close; adaptive compression trades some repair signal for shorter feedback.

Table 2 resolves the apparent baseline mismatch. In Experiment 1, `gpt-4o-mini` execution-only solves 31/92 and `gpt-4o-mini` with `gpt-5-mini` advisory solves 56/92 under a five-round horizon. In Experiment 2, the execution-only baseline solves 25/92 and original advisory solves 46/92 under a tighter two-round horizon and a fixed final prompt design. The final baseline is 25/92 in the authoritative final comparison; I use that value throughout this section. The 56/92 and 46/92 results answer different questions and should not be interpreted as a regression.

After the advisory study showed that semantic feedback can be useful, I compared three ways of using semantic information in the repair loop: full direct advisory, compressed advisory, and `spec_then_code`. Table 4 shows that direct semantic advisory is the strongest final method at 46/92, compared with 25/92 for execution-only. `spec_then_code` is close at 44/92 while using fewer solver calls (154 versus 164) and fewer LLM calls (308 versus 328) than original advisory. Adaptive compression solves 37/92 while substantially reducing prompt and feedback length.

The safety counters are clean in all four rows: expected objective leakage is 0, objective gap leakage is 0, `OBJECTIVE={...}` container output is 0, and output-contract-caused `no_objective` is 0. Parse success is also high for the three semantic methods, ranging from 0.981 to 0.988.

The method overlap suggests that no single semantic packaging is complete. `spec_then_code` solves 9 problems that direct advisory misses, while direct advisory solves 11 that `spec_then_code` misses. Adaptive compression uniquely solves 2 problems, and at least one final method solves 57/92 problems. A practical next system should probably route between direct advisory and formulation planning rather than choose one fixed prompt for every problem.

6.5 Why Not Use the Judge as a Gate?

Judge accuracy explains why I keep the advisor diagnosis-only. Across the advisory study, recall ranges from 0.812 to 1.000, while precision ranges from 0.355 to 0.846. The judge is often permissive: it rarely rejects many correct programs, but it also approves many programs that later fail. The strongest case is `kimi-k2.6` judging `gpt-5-mini` on Band 4, with precision 0.846, recall 1.000, and $F_1 = 0.917$.

Retrospectively, a gate would look attractive on the hardest weak-generator band: for `gpt-4o-mini` on Band 4, it would save 80.2% of executions with 97.9% rejection accuracy. However, it would also falsely reject 15.4% of solved programs. The `gpt-5-mini+kimi-k2.6` Band-4 row is encouraging, with 31.6% saved executions, 100.0% rejection accuracy, and 0.0% false rejection, but it is too narrow to justify a general hard gate. I therefore treat this as a promising cost-control direction, not as permission to block execution.

7 Limitations

Both studies are single-run evaluations at temperature 0.0. The per-band sample sizes are small, especially with 20 problems in the first three bands, so the pooled and paired statistical tests should be read as exploratory. Repeated seeds and a stratified test would make the inference stronger.

The benchmark is also limited. LogiOR is useful because it contains executable evaluator targets, but the result may not transfer to NL4LP, ComplexOR, IndustryOR, or unmasked user problems. The evaluator checks execution behavior and objective agreement; it does not prove that every solved program is semantically correct in a formal sense.

The semantic advisor is itself an LLM. It can miss constraints, approve bad code, or suggest repairs that the generator implements incorrectly. `spec_then_code` can write a wrong spec before writing wrong code. Adaptive compression can remove a detail that was needed for repair. Cost is measured only in API calls, not dollars, latency, or tokens.

Finally, the final system does not include a trained local verifier. The verifier direction remains plausible future work if better paired data and stronger near-miss negatives are available, but it is not a final experimental result in this report.

8 Conclusion

Semantic guidance helps LLM optimization-code repair, but only when it adds useful information. In the cross-difficulty study, advisory raises `gpt-4o-mini` from 31/92 to 56/92 solved problems, with +25 to +30 percentage-point gains in every difficulty band. The paired rescue/loss result, 26 rescues against 1 loss, shows that the gain is not just a change in aggregate rate. The same advisory idea is neutral for `gpt-5-mini`, which moves from 53/92 to 53/92 with same-model advisory and to 54/92 with `kimi-k2.6`.

The final repair comparison turns that finding into a practical design choice. Direct advisory is the strongest method at 46/92, `spec_then_code` is close at 44/92 and solves a complementary subset, and adaptive compression reaches 37/92 while shortening feedback. The efficiency results are encouraging: advisory can reduce API calls per solved hard problem, and compression can make semantic feedback much shorter while still improving over execution-only repair. The conclusion I draw is simple: semantic feedback should be used as diagnosis, not as a hard gate. Execution remains the reliable evaluator, while the advisor helps the generator understand what kind of mathematical model it should have written.

Future work should focus on method routing, calibrated semantic judges, repeated seeds, stratified tests, better verifier data, and selective code-to-spec diagnostics. Interactive optimization work argues that user intent is often clarified through dialogue and structured editing tools rather than a single static prompt, suggesting a natural extension of semantic advisory: convert diagnosis text into user- or tool-editable modeling operations [Drossman et al., 2026].

References

- Ali AhmadiTeshnizi et al. Optimus: Optimization modeling using mip solvers and large language models, 2024. Preprint.
- Nicolas Astorga et al. Autoformulation of optimization models using large language models, 2024. Preprint.
- Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug, 2023. Preprint.
- Joshua Drossman, Alexandre Jacquillat, and Sébastien Martin. Let’s have a conversation: Designing and evaluating LLM agents for interactive optimization, 2026.
- Runda Guan, Xiangqing Shen, Jiajun Zhang, Yifan Zhang, Jian Cheng, and Rui Xia. Execution-verified reinforcement learning for optimization modeling, 2026.
- Gurobi Optimization, LLC. Gurobi optimizer reference manual, 2026. URL <https://www.gurobi.com/documentation/>.
- Chenyang Huang et al. Tree-of-thoughts and monte carlo search for LLM-guided optimization modeling, 2024a. Preprint, LLM-guided search over formulations.

- Xuhan Huang, Qingning Shen, Yan Hu, Anningzhe Gao, and Benyou Wang. LLMs for mathematical modeling: Towards bridging the gap between natural and mathematical languages, 2024b. Findings of NAACL 2025; introduces Mamo benchmark.
- WeiQi Jiang et al. Llmopt: Learning to define and solve general optimization problems from scratch, 2024. Preprint.
- Minwei Kong, Ao Qu, Xiaotong Guo, Wenbin Ouyang, Chonghe Jiang, Han Zheng, Yining Ma, Dingyi Zhuang, Yuhang Tang, Junyi Li, Shenhao Wang, Haris Koutsopoulos, Hai Wang, Cathy Wu, and Jinhua Zhao. AlphaOPT: Formulating optimization programs with self-improving LLM experience library, 2025. Revised 2026.
- Aman Madaan et al. Self-refine: Iterative refinement with self-feedback, 2023. Preprint.
- Shuai Peng, Ke Yuan, Liangcai Gao, and Zhi Tang. MathBERT: A pre-trained model for mathematical formula understanding, 2021. Preprint.
- Rindranirina Ramamonjison et al. Augmenting operations research with auto-formulation of optimization models from problem descriptions, 2022. Preprint / benchmark paper commonly associated with NL4Opt.
- Zhihong Shao et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models, 2024. Preprint introducing Group Relative Policy Optimization (GRPO).
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. Preprint.
- Zhengyang Tang et al. ORLM: A customizable framework in training large models for automated optimization modeling, 2024. Preprint, supervised-fine-tuning approach.
- Alex Wilf, Pranjal Aggarwal, Bryan Parno, Daniel Fried, Louis-Philippe Morency, Paul Pu Liang, and Sean Welleck. Propose, solve, verify: Self-play through formal verification, 2025.
- Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-Pack: Packed resources for general chinese embeddings, 2024a. Preprint introducing the BGE family of retrieval encoders.
- Zhen Xiao et al. Chain-of-experts: When large language models meet complex operations research problems, 2024b. Preprint.
- Beinuo Yang, Qishen Zhou, Junyi Li, Chenxing Su, Panagiotis Angeloudis, and Simon Hu. ORThought: Benchmarking and automating logistics optimization modeling, 2025. Revised 2026.
- Zhicheng Yang, Yiwei Wang, Yinya Huang, Zhijiang Guo, Wei Shi, Xiongwei Han, Liang Feng, Linqi Song, Xiaodan Liang, and Jing Tang. OptiBench meets ReSocratic: Measure and improve LLMs for optimization modeling, 2024. ICLR 2025.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2022. Preprint.
- Xinzhi Zhang, Zeyi Chen, Humishka Zope, Hugo Barbalho, Konstantina Mellou, Marco Molinaro, Janardhan Kulkarni, Ishai Menache, and Sirui Li. OptiMind: Teaching LLMs to think like optimization experts, 2025a. Revised 2026.
- Yansen Zhang, Qingcan Kang, Yujie Chen, Yufei Wang, Xiongwei Han, Tao Zhong, Mingxuan Yuan, and Chen Ma. SAC-Opt: Semantic anchors for iterative correction in optimization modeling, 2025b. Revised 2026.
- Yisong Zhang, Ran Cheng, Guoxing Yi, and Kay Chen Tan. A systematic survey on large language models for evolutionary optimization: From modeling to solving, 2025c. Revised 2026.
- Chenyu Zhou, Tianyi Xu, Jianghao Lin, and Dongdong Ge. StepORLM: A self-evolving framework with generative process supervision for operations research language models, 2025.

A Full Advisory-Study Statistics

The main text reports only the headline advisory pattern. Tables 5 and 6 show the supporting statistical view. The pooled Fisher tests compare aggregate solve counts, while the paired table is more faithful to the experiment because the methods are evaluated on the same 92 problems.

Comparison	Execution-only rate	Advisory rate	Delta	Two-sided Fisher p
gpt-4o-mini exec vs adv(gpt-5-mini)	31/92 (33.70%)	56/92 (60.87%)	+27.17pp	0.0003666
gpt-5-mini exec vs adv(gpt-5-mini)	53/92 (57.61%)	53/92 (57.61%)	+0.00pp	1.0000
gpt-5-mini exec vs adv(kimi-k2.6)	53/92 (57.61%)	54/92 (58.70%)	+1.09pp	1.0000

Table 5: Exploratory pooled Fisher exact tests over all 92 problems.

Comparison	Both solved	Both failed	Rescued by advisory	Lost by advisory	Exact paired p
gpt-4o-mini exec vs adv(gpt-5-mini)	30	35	26	1	4.17×10^{-7}
gpt-5-mini exec vs adv(gpt-5-mini)	49	35	4	4	1.0000
gpt-5-mini exec vs adv(kimi-k2.6)	50	35	4	3	1.0000

Table 6: Paired rescue/loss counts over the same 92 problems.

The paired counts are the clearest evidence for the weak-generator result: advisory creates many more rescues than losses for **gpt-4o-mini**. For **gpt-5-mini**, the rescue and loss counts are balanced, which is why the main paper treats the strong-generator result as neutral rather than as a failed implementation detail.

B Additional Advisory-Study Diagnostics

The presentation report contains several diagnostic views that are too detailed for the main paper but useful for interpreting where semantic advisory changes outcomes. Figure 6 shows all 92 problems sorted by difficulty and makes clear that advisory changes are not concentrated in a single example. Figure 7 summarizes final error categories by band and run, supporting the main claim that objective mismatch is a central semantic failure mode.

The advisor returns both diagnosis text and a binary `should_execute` judgment. Table 7 evaluates that binary judgment retrospectively, using execution validity as the comparison target. This is not a claim that execution validity is a perfect semantic ground truth; it is a practical way to ask whether the advisor is calibrated enough to be trusted as a gate.

Band	Config	Checks	TP	FP	TN	FN	Precision	Recall	F_1
Band 1	gpt-4o-mini adv/gpt-5-mini	56	14	21	19	2	0.400	0.875	0.549
Band 1	gpt-5-mini adv/gpt-5-mini	28	15	7	5	1	0.682	0.938	0.789
Band 1	gpt-5-mini adv/kimi-k2.6	28	13	5	8	2	0.722	0.867	0.788
Band 2	gpt-4o-mini adv/gpt-5-mini	57	12	4	39	2	0.750	0.857	0.800
Band 2	gpt-5-mini adv/gpt-5-mini	27	11	11	4	1	0.500	0.917	0.647
Band 2	gpt-5-mini adv/kimi-k2.6	23	11	6	5	1	0.647	0.917	0.759
Band 3	gpt-4o-mini adv/gpt-5-mini	67	11	20	34	2	0.355	0.846	0.500
Band 3	gpt-5-mini adv/gpt-5-mini	26	14	8	2	2	0.636	0.875	0.737
Band 3	gpt-5-mini adv/kimi-k2.6	28	13	7	5	3	0.650	0.812	0.722
Band 4	gpt-4o-mini adv/gpt-5-mini	121	11	13	95	2	0.458	0.846	0.595
Band 4	gpt-5-mini adv/gpt-5-mini	27	8	6	12	1	0.571	0.889	0.696
Band 4	gpt-5-mini adv/kimi-k2.6	19	11	2	6	0	0.846	1.000	0.917

Table 7: Semantic judge accuracy by band and configuration.

The pattern is high recall with variable precision. In other words, the judge often recognizes candidates that can execute successfully, but it also approves many candidates that later fail. The `kimi-k2.6` Band-4 case is the strongest row, yet the broader table explains why the final system keeps the judgment advisory rather than authoritative.

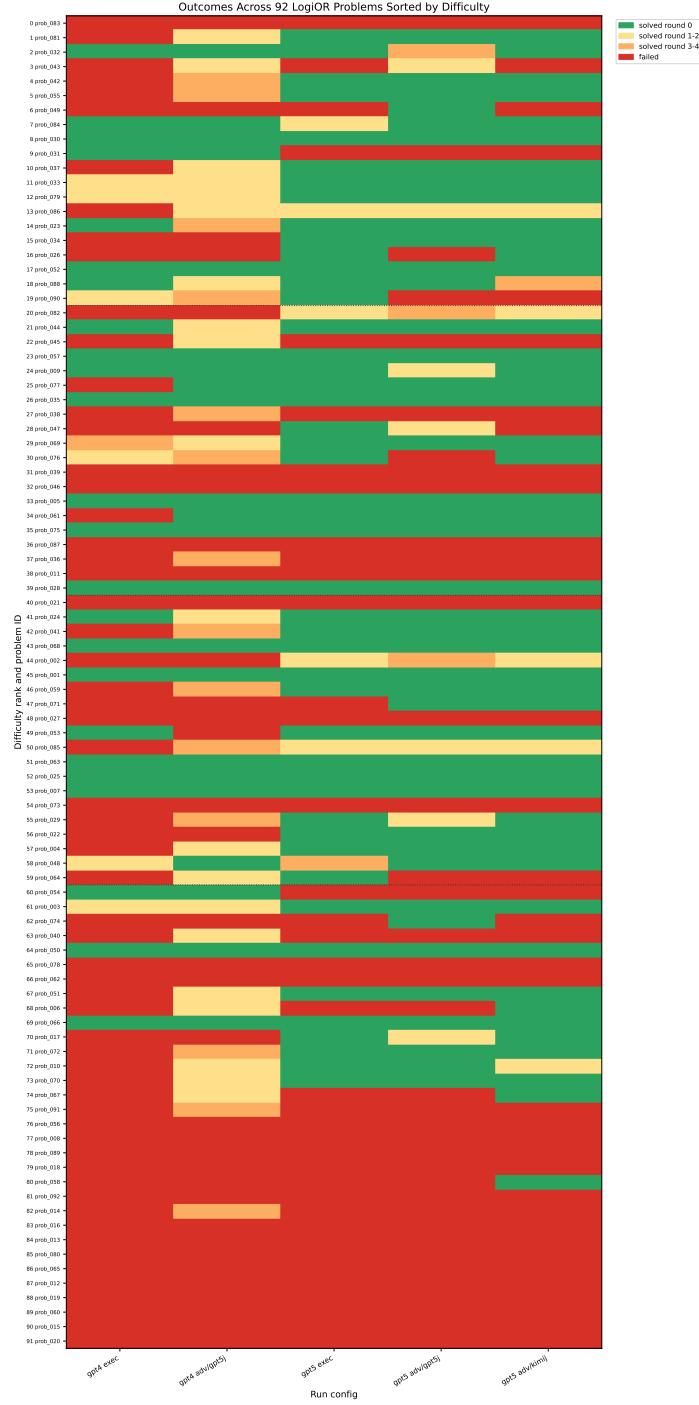


Figure 6: Per-problem advisory-study outcomes across all 92 LogiOR problems sorted by difficulty. This is an overview heatmap rather than a label-by-label lookup table: the important pattern is where outcomes change across configurations and difficulty bands. Rows are problems and columns are configurations/round outcomes; band boundaries mark the four difficulty groups.

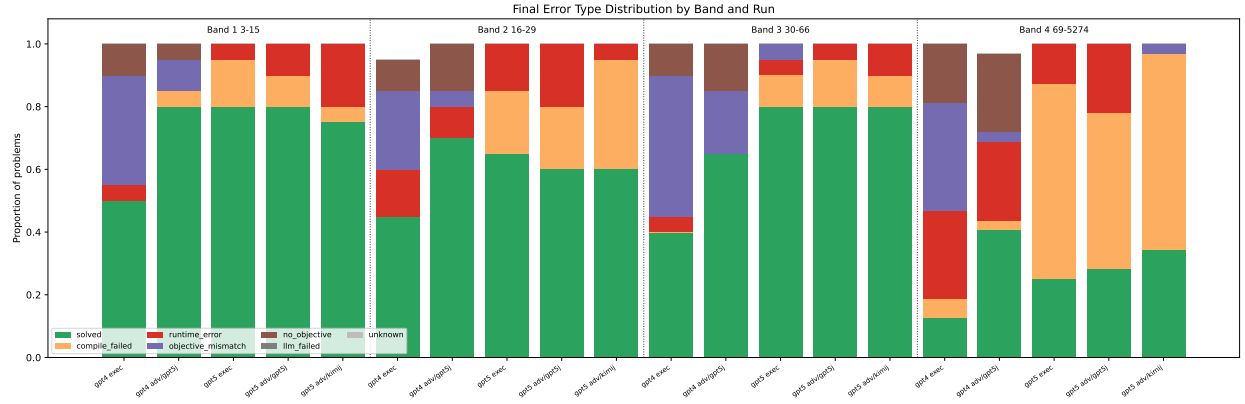


Figure 7: Final error-type distribution by band and run. This diagnostic distribution view is meant to compare failure modes rather than individual instances; the important pattern is the large share of `objective_mismatch` and `no_objective` failures relative to pure compile failures.

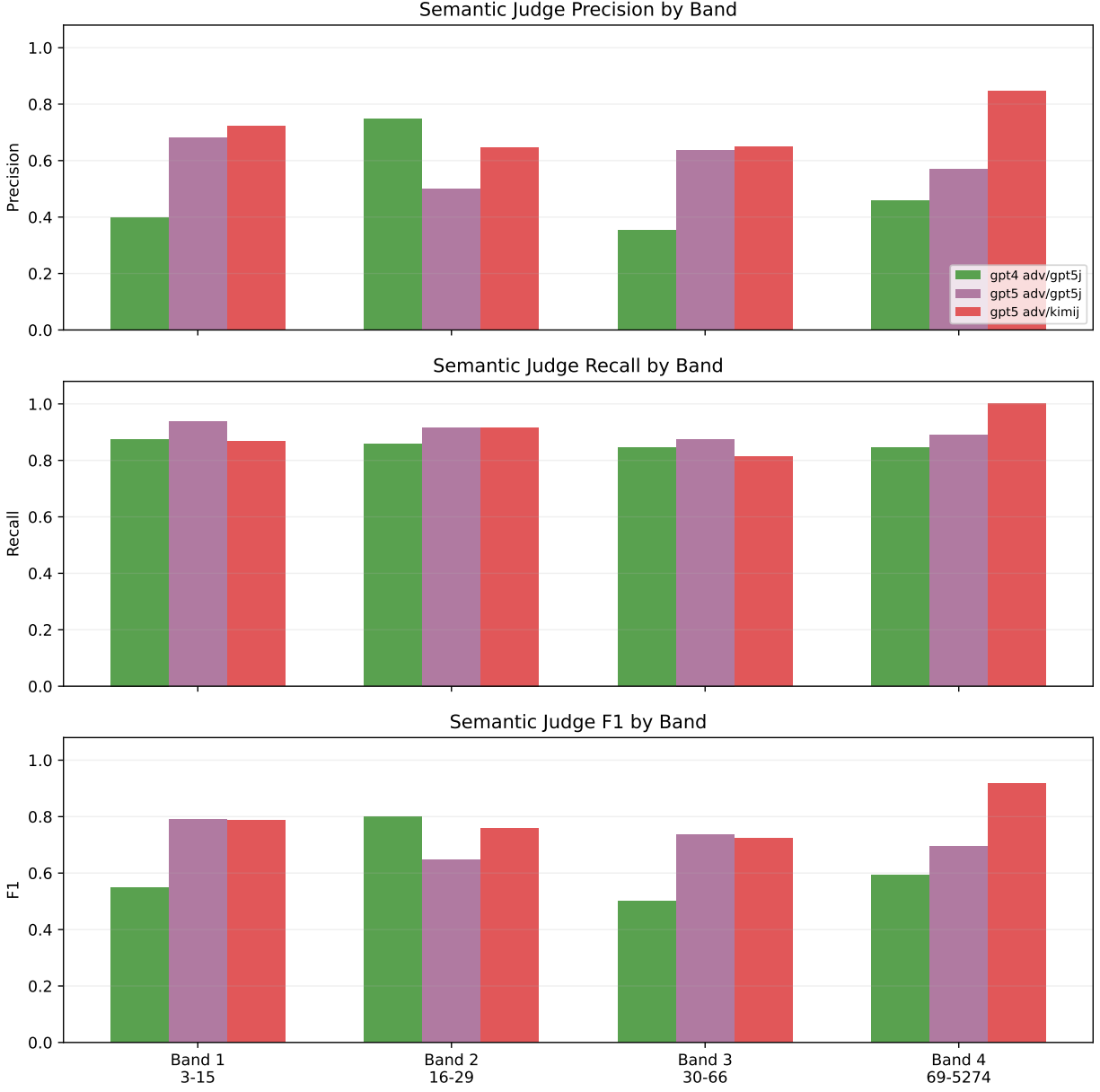


Figure 8: Semantic judge precision, recall, and F_1 by band and configuration in the advisory study. The x-axis groups difficulty bands/configurations and the y-axis is score, where higher is better. The high-recall but variable-precision pattern explains why the advisor is useful as diagnosis text but not reliable enough as an execution gate.

C Gate and Cost Analysis

The following figures and tables collect the operational tradeoffs. They are appendix material because they support the design choice but are not the central result. Figure 9 visualizes saved executions and rejection accuracy for hypothetical gates. Figure 10 shows API calls per solved problem, and Figure 11 summarizes round efficiency.

Metric	Definition	Interpretation
API/Solved	Total API calls divided by number of solved problems.	Lower is better; measures call efficiency.
Overhead vs. exec	API calls of an advisory configuration divided by API calls of the corresponding execution-only configuration on the same band and generator.	Values above 1 mean advisory used more calls than execution-only.
Marginal API/extra solve	(advisory API calls – execution-only API calls) / (advisory solved – execution-only solved).	Incremental API calls required for each additional solved problem.
Max prompt chars	Maximum character length of any LLM prompt in the final comparison.	Lower means shorter prompts; character proxy, not token/dollar cost.
Max feedback chars	Maximum character length of the feedback block inserted into repair prompts, including execution feedback and semantic diagnosis when present.	Lower means less verbose repair context.
Saved %	Fraction of executions that would be skipped if <code>should_execute=false</code> were used as a hard gate.	Higher means more solver-call saving, but only useful if false rejection is low.
Rejection accuracy	Among candidates rejected by the hypothetical gate, fraction that would have failed execution/evaluation.	Higher means the gate rejects mostly bad candidates.
False rejection rate	Fraction of solved candidates that would have been incorrectly rejected by the hypothetical gate.	Lower is safer; nonzero values are why the final method does not block execution.

Table 8: Efficiency and gate metrics.

Band	Config	Checks	Saved %	Rejection Accuracy	Error Capture	False Rejection Rate
Band 1	<code>gpt-4o-mini adv/gpt-5-mini</code>	56	37.5%	90.5%	47.5%	12.5%
Band 2	<code>gpt-4o-mini adv/gpt-5-mini</code>	57	71.9%	95.1%	90.7%	14.3%
Band 3	<code>gpt-4o-mini adv/gpt-5-mini</code>	67	53.7%	94.4%	63.0%	15.4%
Band 4	<code>gpt-4o-mini adv/gpt-5-mini</code>	121	80.2%	97.9%	88.0%	15.4%
Band 4	<code>gpt-5-mini adv/gpt-5-mini</code>	27	48.1%	92.3%	66.7%	11.1%
Band 4	<code>gpt-5-mini adv/kimi-k2.6</code>	19	31.6%	100.0%	75.0%	0.0%

Table 9: Retrospective hard-gate behavior inferred from advisory runs.

Table 9 translates the same issue into a deployment question. A hard gate could save solver calls, but any nonzero false rejection rate means some correct programs would never be executed. That is why the main method logs `should_execute` for analysis while still running every candidate.

Table 10 reports API-call efficiency rather than token or dollar cost. The goal is to separate two effects: advisory adds calls per round, but it can also reduce repeated failed repair attempts by making the feedback more useful.

In Table 10, Overhead vs. exec is the ratio of total API calls for the advisory configuration to total API calls for the matching execution-only configuration in the same band and generator. Marginal API/extra solve is the extra API calls divided by the additional solved problems relative to that execution-only baseline. For Band 4 `gpt-4o-mini`, overhead is $249/145 = 1.7\times$, while marginal API/extra solve is $(249 - 145)/(13 - 4) = 11.6$.

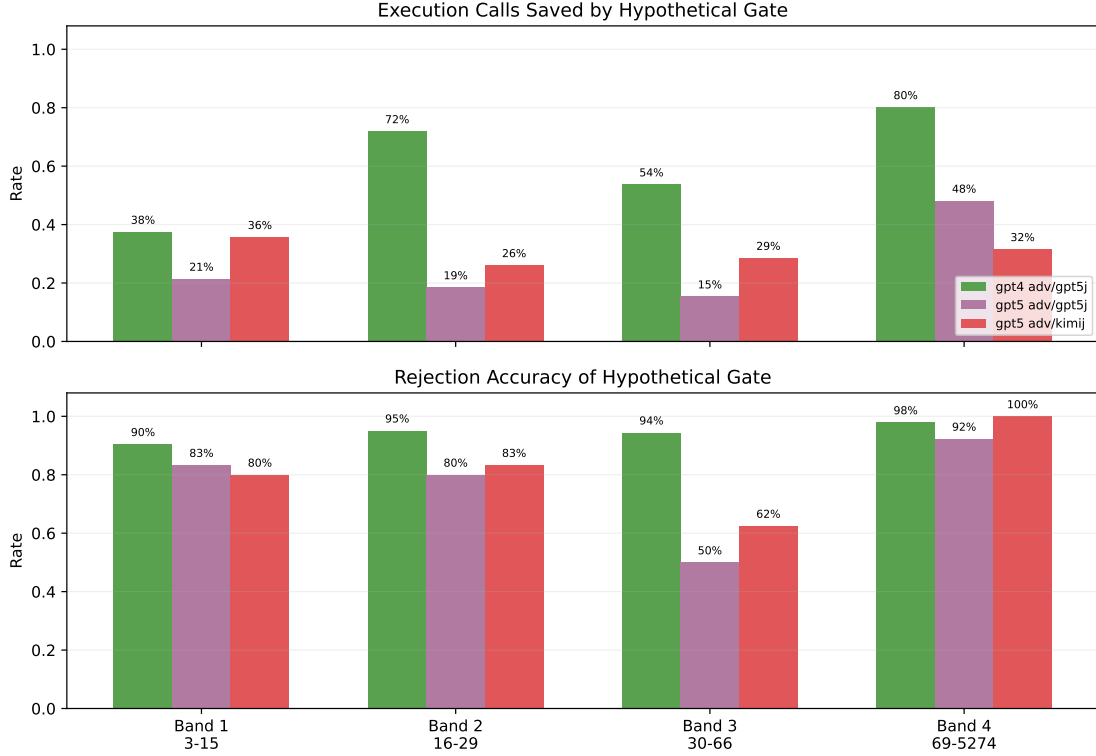


Figure 9: Hypothetical gate efficiency inferred retrospectively from advisory runs. The x-axis is difficulty band/configuration and the y-axis is rate. The top panel shows the fraction of execution calls that would be saved by using `should_execute=false` as a hard gate; higher means more solver-call saving. The bottom panel shows rejection accuracy among skipped candidates; higher means the gate rejects mostly bad candidates. False rejection is reported in Table 9, not shown in the figure, and is why the final method does not block execution.

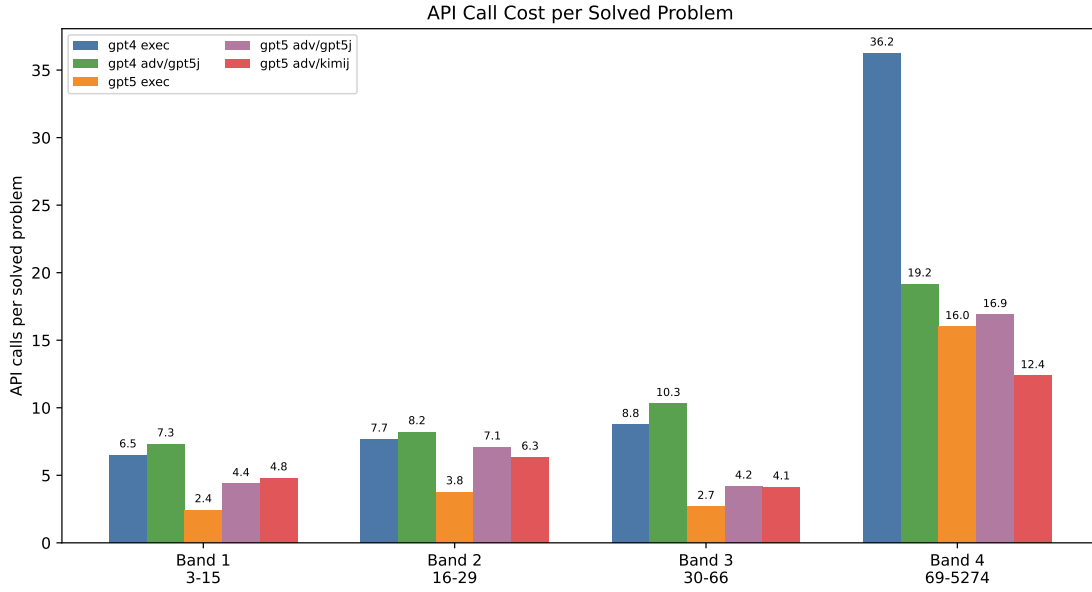


Figure 10: API calls per solved problem in the advisory study. The x-axis is difficulty band/configuration and the y-axis is API calls divided by solved problems, so lower is better. Advisory can improve this metric when added judge calls are outweighed by increased solves; this is a call-count metric, not token or dollar cost.

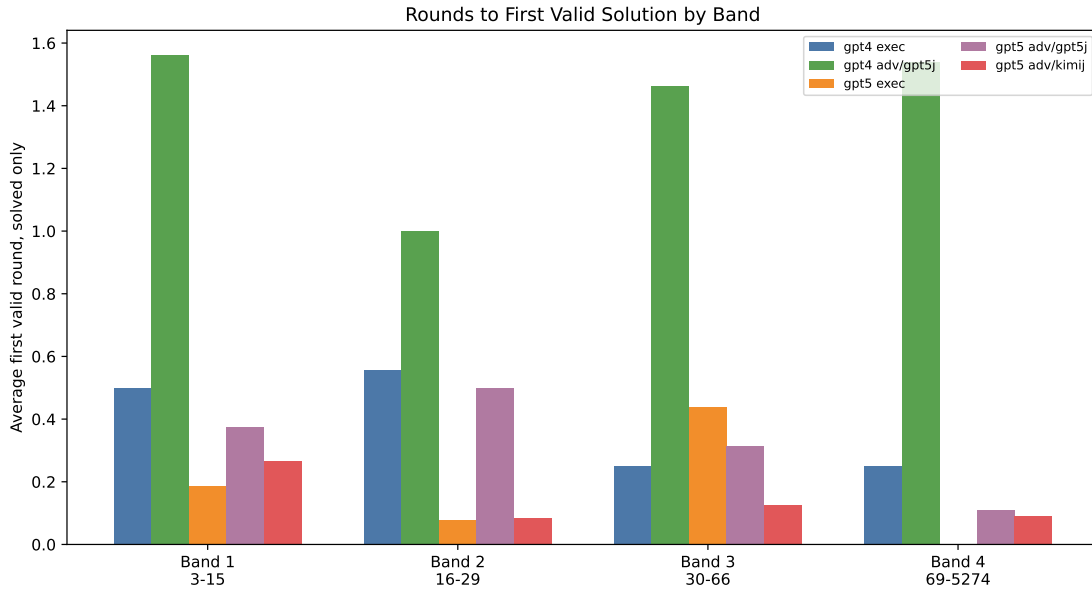


Figure 11: Average rounds to first valid solution for solved problems in the advisory study. The x-axis is difficulty band/configuration and the y-axis is the average first valid round among solved problems, so lower means valid solutions appear earlier. This metric is conditional on solved problems only and is not a solve-rate metric.

Band	Config	Solved	API calls	API/Solved	Overhead vs. exec	Marginal API/extra solve
Band 1	gpt-4o-mini exec	10/20	65	6.5	1.0×	–
Band 1	gpt-4o-mini adv	16/20	117	7.3	1.8×	8.7
Band 1	gpt-5-mini exec	16/20	39	2.4	1.0×	–
Band 4	gpt-4o-mini exec	4/32	145	36.3	1.0×	–
Band 4	gpt-4o-mini adv	13/32	249	19.2	1.7×	11.6
Band 4	gpt-5-mini exec	8/32	128	16.0	1.0×	–
Band 4	gpt-5-mini adv/ kimi-k2.6	11/32	136	12.4	1.06×	2.7

Table 10: Selected API-call cost-efficiency results.

The Band-4 rows are the most important. On the hardest problems, advisory improves calls per solved problem for both the weak-generator setting and the strongest cross-provider setting shown here. This supports the main call-efficiency claim without making a blanket cost claim across all regimes.

D Additional Qualitative Repair Traces

LogiOR prob_037, product-flow repair. **User description.** LogiCorp buys whole units of raw material, produces Basic and Advanced products, and may convert them into Premium Basic and Premium Advanced. The goal is to maximize profit under raw-material and labor limits.

Round 0 generated formulation. The initial generated code had the following essential structure:

```
raw_materials = model.addVar(vtype=GRB.INTEGER)
basic_product = model.addVar(vtype=GRB.INTEGER)
advanced_product = model.addVar(vtype=GRB.INTEGER)
premium_basic = model.addVar(vtype=GRB.INTEGER)
premium_advanced = model.addVar(vtype=GRB.INTEGER)

model.addConstr(basic_product <= 2 * raw_materials)
model.addConstr(advanced_product <= 3 * raw_materials)
model.addConstr(premium_basic <= basic_product)
model.addConstr(premium_advanced <= advanced_product)

model.setObjective(
    10 * basic_product + 12 * advanced_product
    + 20 * premium_basic + 25 * premium_advanced
    - costs,
    GRB.MAXIMIZE)
```

This code constrained conversion only by upper bounds. It did not conserve flow between ordinary products, premium products, and sold units. It also counted revenue from both ordinary and premium products, so converted units were double-counted.

Advisory feedback and execution status. The advisor identified missing flow equalities and double-counted revenue, and recommended sold variables or equality constraints linking production, conversion, and sales. The candidate executed but returned 162,990 instead of 156,500, so it failed with an objective mismatch.

Round 1 code difference. The next repair followed the diagnosis with a small formulation change:

```
+ basic_sold = model.addVar(vtype=GRB.INTEGER, name="basic_sold")
+ advanced_sold = model.addVar(vtype=GRB.INTEGER, name="advanced_sold")
+ model.addConstr(basic_sold + premium_basic == basic_product)
+ model.addConstr(advanced_sold + premium_advanced == advanced_product)
- revenue_basic = 10 * basic_product
- revenue_advanced = 12 * advanced_product
+ revenue_basic = 10 * basic_sold
+ revenue_advanced = 12 * advanced_sold
```

Outcome. Round 1 returned objective 156,500 with zero objective gap. This is the code-level version of the semantic repair example summarized in the main text.

LogiOR prob_081, route-flow repair. **prob_081** is a route-flow problem: ship 1,000 tons from City F to City D over three routes with road and rail segments. The initial **gpt-4o-mini** program used six independent segment variables, so serial segments on the same route could carry different amounts of flow. The first diagnosis identified this route-continuity error and suggested route-flow variables or equality constraints. The next repair fixed continuity but omitted one Route-1 cost term. A second diagnosis flagged the missing $0.80 \cdot 500 \cdot x_1$ term, and the following repair matched the expected objective. This example shows two-step semantic repair: first structural flow feedback, then objective-term feedback.

E Final-System Diagnostics

The following tables summarize design choices that are secondary to the main paper but useful for interpreting the final system. They explain which components became final methods, which remained analysis-only, and why hard gates and uptake metrics should be read cautiously.

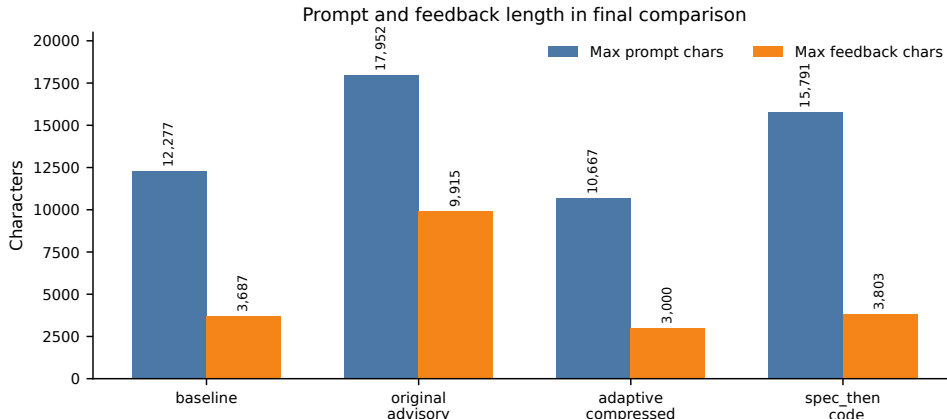


Figure 12: Prompt and feedback length in the final comparison. The x-axis is the final-comparison method and the y-axis is character count, so lower is shorter. Max prompt chars is the maximum character length of any LLM prompt in that method over the full92 final comparison. Max feedback chars is the maximum character length of the feedback block inserted into repair prompts, including execution feedback and semantic diagnosis when present. These are character counts, not token or dollar costs.

Module	Final role	Evidence
baseline	reference	25/92 final comparison
original advisory	strongest repair method	46/92 final comparison
adaptive compressed	prompt-efficiency tradeoff	37/92, max feedback 3000
spec_then_code	structured alternative	44/92, complementary solves
code-to-spec comparison	diagnostic only	useful for analysis, not final repair
hybrid static/spec/semantic	analysis only	noisier than direct advisory
multi-advisor	not cost-effective	higher call cost without better final result
retrospective gate	offline only	false rejects solved code
feedback uptake	analysis only	high implementation, imperfect resolution

Table 11: Role of auxiliary modules in the final system analysis.

Table 11 separates final repair methods from diagnostic modules. This distinction matters because code-to-spec comparison, hybrid diagnosis, multi-advisor disagreement, and feedback uptake helped explain failures, but they were not reliable enough or cost-effective enough to define the final repair loop.

Policy	Saving Rate	False Rejection Rate	False-Rejected Solved
no_gate	0.000	0.000	0
single_should_execute	0.648	0.133	2
confidence_threshold	0.818	0.267	4
reject_only_high_confidence	0.511	0.133	2
static_error_threshold	0.080	0.000	0

Table 12: Retrospective gate policies for the final comparison.

Table 12 repeats the gate lesson under the final comparison setting. More aggressive policies save more executions, but they also reject solved programs. The conservative choice is therefore to use semantic

judgments as repair text, not as execution permission.

Method	Items	Implemented	Resolved	New Error	Gap Improved	Impl.+Solved
adaptive compressed	301	0.967	0.525	0.352	0.183	52
original advisory	283	0.986	0.572	0.431	0.219	40

Table 13: Heuristic offline feedback uptake. Implementation does not imply resolution.

Table 13 measures whether advice was reflected in later code. The high implementation rates show that the generator often tries to follow the advisor. The lower resolution rates show the harder part: implementing a suggestion can introduce a new modeling error or fail to fix the original one.

Category	Count	Problem IDs
spec_then_code unique over original	9	prob_017, prob_024, prob_026, prob_033, prob_041, prob_058, prob_071, prob_081, prob_091
original unique over spec_then_code	11	prob_001, prob_031, prob_034, prob_035, prob_040, prob_043, prob_047, prob_050, prob_064, prob_069, prob_070
adaptive unique over other methods	2	prob_078, prob_086
solved by any final method	57	57/92 total problems solved by at least one method

Table 14: Exact solve-overlap details for the final comparison.

Table 14 gives the problem-level overlap behind the complementarity claim in the main text. Direct advisory and **spec_then_code** each solve problems the other misses, so the natural next step is selective routing rather than choosing one semantic packaging for all inputs.